Smart Pesticide Residue Detection & Multi-Sensor Produce Quality Screening

Appendix: Complete Source Code Architecture & Firmware Implementation Report

Institution: D. Y. Patil College of Engineering & Technology, Kasaba Bawada, Kolhapur
Department: Electronics & Telecommunication Engineering (ENTC)
Academic Year: 2026–2027
Total Source Code: 18,370 Lines of Code across 216 Files

---

1. Lines of Code (LOC) Summary & Metrics

The Smart Pesticide ML system encompasses an end-to-end edge AI pipeline: bare-metal STM32F407 firmware, low-level I2C/ADC/DCMI drivers, TensorFlow Lite for Microcontrollers (TFLM) CMSIS-NN inference, multi-modal sensor fusion algorithms, and model training/quantization toolchains.

```
pie title Codebase Distribution by Language (18,370 Total LOC)
    "Python (ML, Pipelines, Tools, Tests - 12,108 LOC)" : 12108
    "C Source (STM32 App & Drivers - 4,064 LOC)" : 4064
    "C Headers (Protocols & Interfaces - 1,383 LOC)" : 1383
    "Linker & Hardware Config (416 LOC)" : 416
    "Shell Scripts & Toolchains (399 LOC)" : 399
```

1.1 Category Breakdown

Category	File Count	Total Lines of Code	Core Modules & Responsibilities
C Source (`.c`, `.cpp`)	47	**4,064**	Bare-metal MCU runtime, DCMI camera acquisition, sensor drivers, OLED UI, image quality

Category	File Count	Total Lines of Code	Core Modules & Responsibilities
			filter
C Headers (`.h`, `.hpp`)	32	**1,383**	Hardware register maps, board pin configurations, CMSIS-NN descriptors, cycle telemetry structs
Python (`.py`)	105	**12,108**	MobileNetV2 training, INT8 quantization, dataset audit, model 2 logistic regression, regression suite
Shell & Automation (`.sh`, `.bat`)	25	**399**	ARM GCC cross-compilation toolchain, STM32CubeProgrammer flashing, memory benchmarks
Linker & Configs (`.ld`, `.yaml`, `.json`)	7	**416**	Cortex-M4 memory layout, flash sectors, CCMRAM placement, calibration profiles
Total Production Code	**216**	**18,370**	Full software deliverable without external vendor dependencies

---

2. Directory Hierarchy & Code Architecture

```
d:\smart-pesticide-ml\
├── firmware/stm32f407/          [5,476 LOC]  On-device embedded C/C++ firmware & bare-meta
│   ├── app/                     Application cycle orchestrator, UI pagination, sensor fusi
│   ├── board/                   Pinout mappings (DCMI, I2C1, ADC1, USART1, Timers)
│   ├── drivers/                 Low-level hardware drivers (OV2640, TCS34725, DHT22, MQ135
│   ├── platform/                STM32F4 clock trees (168 MHz PLL), DMA2 stream controllers
│   ├── startup/                 Vector table, Reset_Handler, newlib-nano system call stubs
│   ├── linker/                  STM32F407VET6 linker script (512 KB Flash, 128 KB RAM, 64
│   └── tests/                   Host-based GoogleTest / C unit harnesses for firmware logi
├── src/fruitvision/             [1,828 LOC]  Core machine learning training and inference
│   ├── models/                  MobileNetV2-0.35 truncated backbone definitions
│   ├── sensor/                  Model 2 chemical risk model, Nernstian calibration, synthe
│   ├── data/                    AgriFreshNET & FruitNet pipelines, deduplication, burst sp
│   └── validation/              Spatial Laplacian filter, luminance checks, aspect guards
├── scripts/                     [4,532 LOC]  Model training, evaluation, TFLite conversion
├── tools/                       [2,984 LOC]  Live frame capture, serial diagnostic stream,
└── tests/                       [2,842 LOC]  180 automated regression tests verifying pari
```

---

3. Production Embedded Firmware Source Code (STM32F407)

3.1 Main Application Orchestrator (`firmware/stm32f407/app/fv_app.c`)

Orchestrates sensor reads, optical colorimetry, camera DCMI capture, foreign object rejection, and display dispatch.

```
#include "fv_app.h"
#include "fv_sensor.h"
#include "fv_report.h"
#include "fv_result.h"
#include "fv_image_check.h"
#include "fv_common.h"
#include "platform/fv_platform.h"
#include <string.h>

fv_status_t fv_app_init(fv_cycle_t *c) {
    if (!c) return FV_ERR_NULL;
    memset(c, 0, sizeof(*c));
    c->clock_status = fv_platform_clock_init();
    c->oled_status  = fv_platform_oled_init();
    c->tcs_status   = fv_platform_tcs_init();
    c->cam_status   = fv_platform_cam_init();
    fv_report_init(c);
    return FV_OK;
}

fv_status_t fv_app_cycle(fv_cycle_t *c) {
    if (!c) return FV_ERR_NULL;
    c->cycles++;
    c->uptime_ms = fv_platform_millis();

    /* 1. Multi-Sensor Data Acquisition */
    fv_sensor_record_t rec;
    memset(&rec, 0, sizeof(rec));
    c->last_sensor_status = fv_sensor_read_all(c, &rec);

    /* 2. Optical TCS34725 Presence Check */
    c->fruit_present = fv_sensor_presence_detect(c, &rec);

    /* 3. Camera DCMI Capture & Image Consistency */
    if (c->fruit_present) {
```

```
        c->last_capture_status = fv_platform_cam_capture(c->frame_buf, sizeof(c->frame_buf)
        if (c->last_capture_status == FV_OK) {
            c->camera_status = FV_OK;
            c->image_detail = fv_image_calculate_detail(c->frame_buf, 160, 120);

            /* Run MobileNetV2 CMSIS-NN Inference */
            fv_vision_run(c->frame_buf, &c->image);
        }
    }

    /* 4. Multi-Layer Non-Fruit Foreign Object Rejection Gate */
    if (c->fruit_present && (c->camera_status == FV_OK)) {
        /* High texture bound (>1400.0) filters leather wallets, keys, fabrics, hands */
        if (c->image.fruit_conf < 0.35f || c->image_detail > 1400.0f) {
            c->image_flags |= FV_IC_REJ_SKIN_OBJECT;
            c->fruit_present = 0;
            c->image.fruit_state = FV_STATE_UNCERTAIN;
            c->image.fruit = -1;
        } else if (rec.has_tcs && (rec.tcs_b * 10u > rec.tcs_r * 35u) && (rec.tcs_b * 10u >
            /* Synthetic Blue Pigment Rejection */
            c->image_flags |= FV_IC_REJ_SKIN_OBJECT;
            c->fruit_present = 0;
            c->image.fruit_state = FV_STATE_UNCERTAIN;
            c->image.fruit = -1;
        }
    }

    /* 5. Optical Species Disambiguation (Chlorophyll & Anthocyanin Ratios) */
    if (c->fruit_present && c->image.fruit == 0 /* Apple */ && rec.has_tcs) {
        if ((rec.tcs_g * 10u > rec.tcs_r * 18u) && (rec.tcs_g >= rec.tcs_b)) {
            c->image.fruit = 2; /* Guava chlorophyll peak reclassification */
        }
    }

    /* 6. Chemical Risk Fusion & Origin Profile Normalization */
    if (c->fruit_present) {
        fv_risk_predict(&rec, &c->risk);
        fv_risk_apply_origin_profile(c->image.fruit, &c->risk);
    }

    /* 7. OLED Screen Update (Spacious 8-Row Layout) */
    fv_report_render_oled(c);
    return FV_OK;
}
```

3.2 Dynamic 8-Row OLED UI Renderer (`firmware/stm32f407/app/fv_report.c`)

Renders high-visibility, spaced information pages on the 128×64 SSD1306 display.

```
#include "fv_report.h"
#include "drivers/fv_ssd1306.h"
#include <stdio.h>
#include <string.h>

static uint8_t s_page_index = 0;
static const char *s_fruit_names[] = {"Apple", "Banana", "Guava", "Tomato", "Brinjal"};

void fv_report_render_oled(const fv_cycle_t *c) {
    fv_ssd1306_clear();

    if (!c->fruit_present) {
        if (c->image_flags & FV_IC_REJ_SKIN_OBJECT) {
            /* Foreign Object / Wallet Rejection Screen */
            fv_ssd1306_draw_string(0, 0,  "====================");
            fv_ssd1306_draw_string(0, 10, " OPTICAL AI SCANNER  ");
            fv_ssd1306_draw_string(0, 20, "====================");
            fv_ssd1306_draw_string(0, 30, " [ INVALID SAMPLE ]  ");
            fv_ssd1306_draw_string(0, 40, " Non-Fruit / Hand!   ");
            fv_ssd1306_draw_string(0, 48, " Target Dist:10-12cm ");
            fv_ssd1306_draw_string(0, 56, " >> SCAN REJECTED << ");
        } else {
            /* Standby Screen */
            fv_ssd1306_draw_string(0, 0,  " SMART PESTICIDE ML  ");
            fv_ssd1306_draw_string(0, 16, " D.Y.PATIL ENTC DEPT ");
            fv_ssd1306_draw_string(0, 32, " Place Fruit On Tray ");
            fv_ssd1306_draw_string(0, 48, " Optical Sensor Ready");
        }
        fv_ssd1306_update();
        return;
    }

    char line[24];
    s_page_index = (c->cycles / 2) % 4;

    switch (s_page_index) {
        case 0: /* Page 1: Fruit Identification & Condition */
            fv_ssd1306_draw_string(0, 0,  "=== 1. AI SPECIES ===");
            snprintf(line, sizeof(line), " Fruit: %s", s_fruit_names[c->image.fruit]);
            fv_ssd1306_draw_string(0, 16, line);
```

```
            snprintf(line, sizeof(line), " Conf:  %.1f %%", c->image.fruit_conf * 100.0f);
            fv_ssd1306_draw_string(0, 32, line);
            snprintf(line, sizeof(line), " State: %s", c->image.condition == 0 ? "Fresh" :
            fv_ssd1306_draw_string(0, 48, line);
            break;

        case 1: /* Page 2: Quality Grade & Shelf Life */
            fv_ssd1306_draw_string(0, 0,  "=== 2. FRESHNESS ====");
            snprintf(line, sizeof(line), " Freshness: %d %%", c->image.fresh_pct);
            fv_ssd1306_draw_string(0, 16, line);
            snprintf(line, sizeof(line), " Quality:   %s", c->image.quality_str);
            fv_ssd1306_draw_string(0, 32, line);
            snprintf(line, sizeof(line), " Shelf:     %d Days", c->image.shelf_days);
            fv_ssd1306_draw_string(0, 48, line);
            break;

        case 2: /* Page 3: Chemical Sensor Telemetry */
            fv_ssd1306_draw_string(0, 0,  "=== 3. SENSOR DATA ==");
            snprintf(line, sizeof(line), " Temp/RH: %.1fC  %d%%", c->temp_c, (int)c->humidi
            fv_ssd1306_draw_string(0, 16, line);
            snprintf(line, sizeof(line), " MQ135:   %d counts", c->mq135_raw);
            fv_ssd1306_draw_string(0, 32, line);
            snprintf(line, sizeof(line), " Wash pH: %.2f", c->ph_value);
            fv_ssd1306_draw_string(0, 48, line);
            break;

        case 3: /* Page 4: Chemical Residue Risk & Safety Badge */
            fv_ssd1306_draw_string(0, 0,  "=== 4. RISK ALERT ===");
            if (c->risk.level == FV_RISK_LOW) {
                fv_ssd1306_draw_string(0, 16, " Risk: LOW (Organic) ");
                fv_ssd1306_draw_string(0, 32, " Safe Lycopene Salad ");
                fv_ssd1306_draw_string(0, 48, " >> SAFE TO EAT <<   ");
            } else {
                fv_ssd1306_draw_string(0, 16, " Risk: MODERATE      ");
                fv_ssd1306_draw_string(0, 32, " Market Wax / Pestic ");
                fv_ssd1306_draw_string(0, 48, " >> WASH REQUIRED << ");
            }
            break;
    }
    fv_ssd1306_update();
}
```

3.3 Sensor Drivers & Analog Subsystems

TCS34725 Optical Colorimeter Driver (`firmware/stm32f407/drivers/fv_tcs34725.c`)

```
#include "fv_tcs34725.h"
#include "fv_bitbang_bus.h"

#define TCS_ADDR 0x29
#define TCS_ENABLE 0x00
#define TCS_ATIME  0x01
#define TCS_CONTROL 0x0F
#define TCS_CDATAL 0x14

fv_status_t fv_tcs34725_init(void) {
    /* 50ms Integration time (ATIME = 0xEB), 4x Gain */
    fv_i2c_write_reg(TCS_ADDR, 0x80 | TCS_ATIME, 0xEB);
    fv_i2c_write_reg(TCS_ADDR, 0x80 | TCS_CONTROL, 0x01);
    /* Power ON and RGBC ADC Enable */
    fv_i2c_write_reg(TCS_ADDR, 0x80 | TCS_ENABLE, 0x03);
    return FV_OK;
}

fv_status_t fv_tcs34725_read_channels(uint16_t *c, uint16_t *r, uint16_t *g, uint16_t *b) {
    uint8_t buf[8];
    fv_i2c_read_bytes(TCS_ADDR, 0x80 | 0x20 | TCS_CDATAL, buf, 8);
    *c = (uint16_t)(buf[1] << 8 | buf[0]);
    *r = (uint16_t)(buf[3] << 8 | buf[2]);
    *g = (uint16_t)(buf[5] << 8 | buf[4]);
    *b = (uint16_t)(buf[7] << 8 | buf[6]);
    return FV_OK;
}
```

DHT22 Thermo-Hygrometric Driver (`firmware/stm32f407/drivers/fv_dht22.c`)

```
#include "fv_dht22.h"
#include "fv_gpio.h"
#include "platform/fv_platform.h"

fv_status_t fv_dht22_read(float *temp_c, float *humidity) {
    uint8_t data[5] = {0};
    /* 1. Host low start pulse (1.2 ms) */
    fv_gpio_set_mode(DHT_PIN, GPIO_OUTPUT);
    fv_gpio_write(DHT_PIN, 0);
    fv_platform_delay_us(1200);
    fv_gpio_write(DHT_PIN, 1);
```

```
    fv_platform_delay_us(30);
    fv_gpio_set_mode(DHT_PIN, GPIO_INPUT);

    /* 2. Read 40 bits via pulse-width timing */
    for (int i = 0; i < 40; i++) {
        while (!fv_gpio_read(DHT_PIN));
        uint32_t t_start = fv_platform_micros();
        while (fv_gpio_read(DHT_PIN));
        if ((fv_platform_micros() - t_start) > 40) {
            data[i / 8] |= (1 << (7 - (i % 8)));
        }
    }

    /* 3. Checksum verification */
    if (((data[0] + data[1] + data[2] + data[3]) & 0xFF) != data[4]) {
        return FV_ERR_CHECKSUM;
    }

    *humidity = ((data[0] << 8) | data[1]) * 0.1f;
    *temp_c   = (((data[2] & 0x7F) << 8) | data[3]) * 0.1f;
    if (data[2] & 0x80) *temp_c = -*temp_c;
    return FV_OK;
}
```

Nernstian Glass-Electrode pH Driver (`firmware/stm32f407/drivers/fv_entc_ph.c`)

```
#include "fv_entc_ph.h"

float fv_ph_calculate(uint16_t adc_counts, float temp_c) {
    /* STM32 12-bit ADC: 3.3V reference voltage */
    float voltage = ((float)adc_counts * 3.3f) / 4095.0f;
    /* Nernst slope adjusted for solution temperature: 0.05916 V/pH @ 25C */
    float nernst_slope = 0.0542f + (0.000198f * (temp_c + 273.15f));
    /* Neutral reference calibration: 2.05 V @ pH 7.00 */
    float ph = 7.00f + ((2.05f - voltage) / nernst_slope);
    if (ph < 0.0f) ph = 0.0f;
    if (ph > 14.0f) ph = 14.0f;
    return ph;
}
```

---

4. Machine Learning & Training Pipeline (Python)

4.1 MobileNetV2 INT8 Model Architecture (src/fruitvision/models/mobilenet.py)

```
import tensorflow as tf
from tensorflow.keras import layers, Model

def create_model1_mobilenet(input_shape=(80, 80, 3), alpha=0.35, cut_layer='block_15_add'):
    """
    Constructs truncated MobileNetV2-0.35 for STM32F407 deployment.
    Reduces parameter memory from 589 KB to 326 KB to fit 512 KB Flash ROM.
    """
    inputs = layers.Input(shape=input_shape, name='image_input')

    # In-model normalization layer [-1, 1]
    norm = layers.Rescaling(scale=1.0/127.5, offset=-1.0, name='normalize')(inputs)

    # Backbone initialized from ImageNet weights
    base_model = tf.keras.applications.MobileNetV2(
        input_tensor=norm,
        alpha=alpha,
        weights='imagenet',
        include_top=False
    )

    # Truncate at cut_layer to minimize peak activation tensors
    backbone_output = base_model.get_layer(cut_layer).output
    x = layers.GlobalAveragePooling2D(name='global_avg_pool')(backbone_output)
    x = layers.Dropout(0.2, name='dropout')(x)

    # Head 1: 5-Class Fruit Identification Softmax
    fruit_out = layers.Dense(5, activation='softmax', name='fruit')(x)

    # Head 2: Binary Freshness Condition Softmax (Fresh / Spoiled)
    cond_out = layers.Dense(2, activation='softmax', name='condition')(x)

    return Model(inputs=inputs, outputs=[fruit_out, cond_out], name='FruitVision_Model1')
```

4.2 Model 2: Sensor Fusion Chemical Risk Classifier (scripts/model2_train.py)

```
import numpy as np
from sklearn.linear_model import LogisticRegression
```

```
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import classification_report, f1_score

def train_sensor_fusion_model(train_csv_path, val_csv_path):
    """
    Fits multinomial logistic regression engine on 12 physical and chemical features.
    Achieves 78.6% test accuracy and embeds within 252 bytes on Cortex-M4.
    """
    # Features: [tcs_r, tcs_g, tcs_b, tcs_c, mq135, ph, temp, rh, r/c, g/c, b/c, ln(mq135)]
    X_train, y_train = load_sensor_features(train_csv_path)
    X_val, y_val = load_sensor_features(val_csv_path)

    scaler = StandardScaler()
    X_train_scaled = scaler.fit_transform(X_train)
    X_val_scaled = scaler.transform(X_val)

    clf = LogisticRegression(
        multi_class='multinomial',
        solver='lbfgs',
        max_iter=1000,
        C=1.0,
        random_state=42
    )
    clf.fit(X_train_scaled, y_train)

    preds = clf.predict(X_val_scaled)
    print("Validation Classification Report:")
    print(classification_report(y_val, preds, target_names=['LOW', 'MEDIUM', 'HIGH']))

    # Export parameters for embedded C integration (fv_sensor.c)
    weights = clf.coef_
    biases = clf.intercept_
    means = scaler.mean_
    scales = scaler.scale_
    return weights, biases, means, scales
```

---

5. Verification & Test Suite Summary

The software repository features 18 test modules totaling **2,842 lines of test code** verifying hardware-software parity: 1. `tests/test_fw_stm32_app.py`: Full system integration test with mock hardware peripherals. 2. `tests/test_fw_presence_gate.py`: Rejection of foreign non-fruit textures (wallets, cards). 3. `tests/test_model2_export_parity.py`: Verifies bit-identical predictions between Scikit-Learn and embedded C linear algebra. 4.

`tests/test_fw_camera_160x120.py`: Direct memory access (DMA) frame alignment and stride tests.

Every test in the test suite passes on both Windows host and ARM GCC target environments.